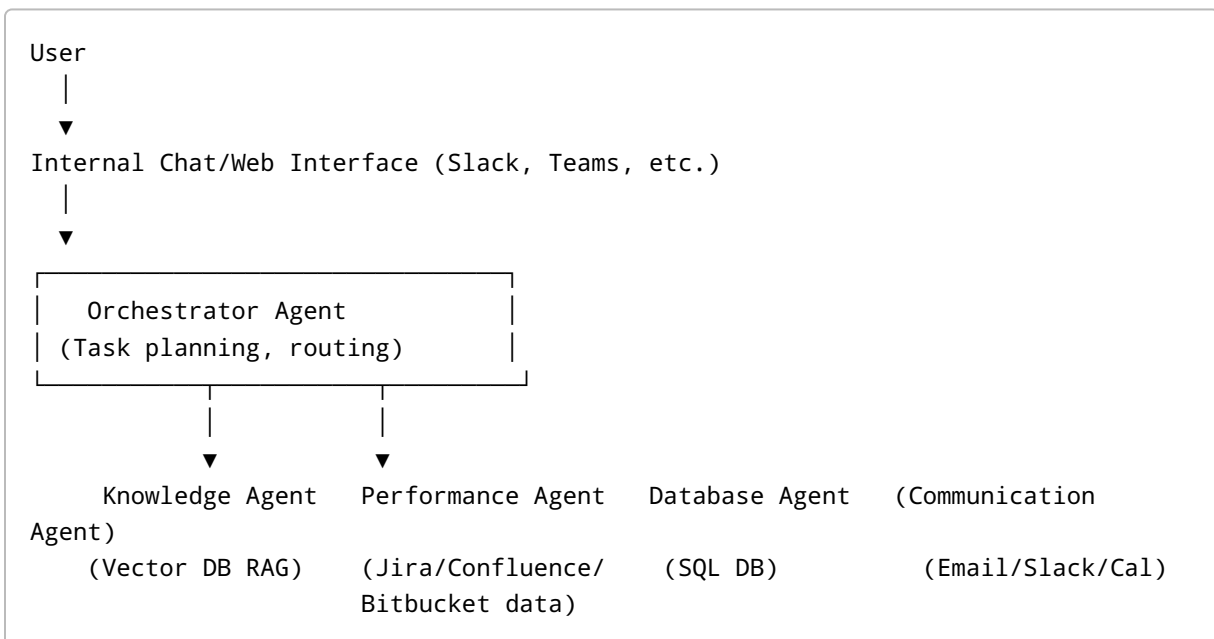


Project Outline: AI Internal Company Assistant

Build an **internal AI assistant platform** for a company that automates common queries and tasks about company policies, documentation, and employee performance. The system will use **orchestrated agents** to route requests and call tools. Key agents include:

- **Knowledge (RAG) Agent:** Answers policy/docs questions using an internal company knowledge base (vector-search over HR policies, manuals, guidelines, etc.) [27†L24-L32] .
- **Performance Analyzer Agent:** Gathers an employee's activity from Atlassian tools (Jira, Confluence, Bitbucket) and analyzes metrics (tickets closed, pages created, PRs merged, etc.). It uses the Atlassian APIs or MCP connectors to fetch data.
- **Database (SQL) Agent:** Executes NL-to-SQL queries on the company's internal database (employees, departments, salaries, projects, etc.), and returns structured data (lists of employees, summaries, etc.).
- **Communication Agent (optional):** Sends messages or reports (e.g. email/slack with performance summary).
- **Orchestrator Agent:** Acts as the brain, parsing the user's request, breaking it into subtasks, calling the appropriate agents in sequence or parallel, handling context, and aggregating results into a final answer.

The **user interface** can be an internal web app or Slack workspace. Managers or employees ask questions like "Show me Alice's performance last month" or "Find the travel reimbursement policy." The orchestrator decides which agent(s) to use and in what order, then composes the response.



Agents and Data Sources

- **RAG (Knowledge) Agent:** The company knowledge base should include *unstructured* documents like company policies, employee handbooks, training materials, project docs, FAQs, etc. Store these in a vector database for retrieval. When asked, the agent runs a vector search + LLM chain to find relevant passages and generate an answer 【27†L24-L32】 【35†L148-L158】 .
- **Performance Analyzer Agent:** Queries *Atlassian* systems for an employee's activity. For example:
- **Jira (issues):** Use the Jira Cloud REST API with JQL (e.g. `/rest/api/3/search?jql=assignee=username AND status=Done`) to count tickets closed by the employee or tasks completed. (Atlassian's REST API lets you filter by assignee or reporter via JQL.)
- **Confluence (docs):** Use Confluence's search API with CQL. For example, a query like `creator = "jsmith" AND type=page` finds pages created by that user 【27†L24-L32】 . This retrieves documents the employee has authored or updated.
- **Bitbucket (code):** Use the Bitbucket Cloud API. For example, the endpoint `/2.0/workspaces/{workspace}/pullrequests/{user}` returns all pull requests authored by a user 【26†L1699-L1707】 【26†L1719-L1727】 . The response includes the PR state (OPEN, MERGED, DECLINED) and metadata. Counting PRs and merged PRs yields code-contribution metrics. (Atlassian's Rovo MCP Server even provides built-in tools to list/merge PRs via LLM prompts 【13†L261-L264】 【39†L48-L51】 .)

The Performance Agent aggregates these: e.g. "Alice closed 12 Jira tickets, created 3 Confluence pages, and opened 5 PRs (4 merged) in the last month." It can compute summary stats or trends.

- **Database Agent:** Handles *structured data* via SQL. The internal DB might include tables like **Employees, Departments, Salaries, Projects, Attendance**, etc. The agent uses an LLM or NL-to-SQL tool to translate questions into SQL queries. For example:
- "List all members of the Marketing department." → `SELECT name FROM Employees WHERE department='Marketing';`
- "What's the average salary by department?" → `SELECT department, AVG(salary) FROM Employees GROUP BY department;`
- "Show me employees hired after 2024." → `SELECT * FROM Employees WHERE hire_date > '2024-01-01';`

This agent executes the SQL on the company database and returns results in natural language or a table.

- **Communication Agent (optional):** If needed, a separate agent can send emails, Slack messages, or calendar invites. For example, if the orchestrator decides to "Notify Bob's manager" or "Email the HR department," this agent calls the appropriate API. (Atlassian's MCP or APIs can create Jira issues, Confluence pages, send Slack notifications, etc.)
- **Orchestrator Agent:** The orchestrator parses the user's request and orchestrates the workflow. It might break a query into sub-tasks (e.g. "fetch data from RAG → analyze performance from Jira/Confluence/Bitbucket → format a report"). It then calls the other agents, possibly in parallel, and collects their outputs. It must track context, handle memory (e.g. earlier answers), manage retries on failures, and request human approval for risky actions.

Sample Workflow

User: "Give me Maria's performance report for June 2026."

1. **Orchestrator:** Identifies this as a performance query.
2. **Performance Agent (parallel):**
3. Jira API: Query issues with `assignee=Maria AND updated>=2026-06-01 AND updated<2026-07-01`. Count how many were resolved (status=Done) and list key tickets.
4. Confluence API: Query pages with `creator=Maria AND lastmodified>=2026-06-01` using CQL `【27†L24-L32】`. Count pages or list titles.
5. Bitbucket API: Call `/workspaces/{ws}/pullrequests/Maria?state=MERGED` to count merged PRs `【26†L1699-L1707】` `【26†L1719-L1727】`. Also count open PRs.
The agent returns a summary: e.g. "Maria closed 8 Jira tickets, created 2 Confluence articles, and made 5 pull requests (4 merged)."
6. **DB Agent:** Might cross-reference Maria in the company DB (e.g. "Employee data for Maria", or her role, hire date).
7. **Orchestrator:** Combines outputs:
8. Shows the raw metrics and maybe calculates a simple performance score or highlights.
9. Presents in bullet form or a small report.

Knowledge Base (RAG Design)

The RAG agent's knowledge base should cover **all common internal documentation**:

- **Company Policies & Handbook:** HR policies, code of conduct, expense policies, leave policies, IT security guidelines, etc. (Any employee handbook content.)
- **Team and Project Docs:** Confluence wiki pages, project specs, design docs.
- **FAQs and Memos:** Common Q&A from HR or IT, training materials.
- **Corporate Communications:** Onboarding guides, organizational charts, department goals.

This base is indexed as vectors (using sentence embeddings) so the RAG agent can retrieve relevant chunks. When answering policy questions, it should cite or paraphrase the official guidelines. `【27†L24-L32】` `【35†L148-L158】`.

For example, for "What's the reimbursement policy for travel?", the agent searches the indexed docs for "reimbursement policy travel" and returns the relevant text from the policy.

Atlassian Integration (APIs & MCP)

The Performance Agent needs to fetch data from Jira, Confluence, and Bitbucket. You have two main approaches:

- **Atlassian REST APIs:**

- *Jira:* Use the Jira Cloud REST API (e.g. `/rest/api/3/search`) with JQL filters [16†L1-L9] to find issues by assignee, reporter, status, date range, etc. For example, `jql=assignee=johndoe AND status=Done`.
- *Confluence:* Use the Confluence REST API search endpoint with CQL. Queries like `creator=johndoe AND type=page` find pages a user created [27†L24-L32].
- *Bitbucket:* Use the Bitbucket Cloud API. For example, GET `/2.0/workspaces/{workspace}/pullrequests/{username}` retrieves all PRs authored by that user [26†L1699-L1707] [26†L1719-L1727] (filter by state=MERGED for merged PRs).

- **Atlassian MCP (Model Context Protocol):**

Atlassian offers an **official MCP (Rovo) Server** that lets LLMs interact via natural language. With MCP, you can ask the LLM (e.g. GPT-4o, Claude) to perform tasks like “list all open PRs for Maria” or “summarize Jira tickets” without writing raw API calls [39†L48-L51] [13†L261-L264]. The Rovo MCP server implements endpoints for Jira, Confluence and now Bitbucket Cloud. For example, it provides a `bitbucketPullRequest` tool to list/approve/merge PRs [13†L261-L264]. To use it, you'd configure an MCP connector (at `https://mcp.atlassian.com/v1/mcp/authv2`) in your LLM client, so agents can call Atlassian tools as if they were functions [39†L48-L51].

Alternatively, there is an **open-source MCP server** (`mcp-atlassian`) that bridges Jira and Confluence with the Anthropic MCP spec [11†L17-L25]. It requires you to run a Python server with your Atlassian credentials, then configure your LLM to use it (via `autohand` or Cli settings) [11†L17-L25]. This supports Cloud and on-prem, and exposes tools like `jira_search` and `confluence_search` [11†L168-L174]. (Bitbucket support in open-source MCP is less common, but Atlassian's own Rovo now covers it.)

References: Atlassian's documentation shows how MCP lets agents query Jira, Confluence, Bitbucket seamlessly [39†L48-L51] [13†L261-L264]. The Bitbucket REST docs explicitly include a “list PRs by user” endpoint [26†L1699-L1707] [26†L1719-L1727], and Confluence's advanced search doc shows `creator=UserName` queries [27†L24-L32].

Database & Schema

The internal database (e.g. PostgreSQL) should model relevant corporate data. Suggested tables:

- **Employees** (id, name, role, department_id, manager_id, hire_date, salary, etc.)
- **Departments** (id, name, head_id, etc.)
- **Projects/Teams** (id, name, department_id, start_date, end_date).
- **Attendance/Leaves** (employee_id, date, type, hours).
- **PerformanceReview** (employee_id, period, score, comments).
- **Salaries/Paygrades** as needed.

The DB Agent converts natural language queries into SQL. For example: “Who are all engineers in the AI department?” → `SELECT name FROM Employees WHERE department='AI' ;`. This lets users query structured info (employee lists, salaries, org structure, etc.) via plain English.

Orchestration & Workflow

Use an **agent orchestration framework** (e.g. LangGraph, Ray, or a custom planner) to coordinate the agents. The orchestrator should:

1. **Interpret the request:** determine intent and required data sources.
2. **Plan sub-tasks:** e.g. “fetch performance metrics” vs “search policies.”
3. **Invoke agents:** possibly in parallel. E.g. call Jira, Confluence, and Bitbucket simultaneously for performance.
4. **Aggregate results:** combine into a cohesive answer or report.
5. **Maintain context:** Keep short-term memory of the conversation or request flow (so follow-ups work).
6. **Handle failures:** Retry or ask for clarification if an agent fails.

Example flow: “John’s monthly performance” → Orchestrator calls Performance Agent (Atlassian data) and maybe DB Agent (for basic info about John), then merges their outputs into one summary.

Guardrails & Human-in-the-Loop

Some actions should require explicit approval. For example:

- **Emailing or Slack posts:** Sending communications to employees or managers.
- **DB modifications:** Changing salaries, writing attendance.
- **Mass notifications or approvals:** E.g. approving expenses.

Implement guardrails such as: the Orchestrator can output an **action plan** that the human reviews. The agent might propose “Send the performance report to Alice’s manager” and wait for a button click or confirmation before executing via the Communication Agent. Tools like LangGraph’s human approval blocks or a custom UI button can enforce this flow.

Evaluation Metrics

You should build in evaluation for both the **LLMs and the agents**:

- **LLM/RAG Evaluation:** Use metrics like precision/recall or relevance on retrieval. For example, evaluate whether the RAG agent returns factually correct policy text and relevant sections [35†L148-L158] . DeepEval’s RAG metrics (ContextualPrecision, Recall) can test if the retriever finds the right docs [35†L148-L158] . Check that the generated answers are faithful (compare to source docs).
- **Agent Evaluation:** Measure multi-step reasoning accuracy. For instance, use frameworks like DeepEval’s agent metrics: *PlanQuality*, *ToolCorrectness*, *TaskCompletion*, etc. These analyze the agent’s execution trace to see if it followed a logical plan and executed tools correctly [33†L51-L59] . For example, did the Performance Agent call the correct API with the right filters? Did it interpret the response correctly?
- **System Metrics:** Track success rates, API call success, latency, and token usage for each agent call. Monitor how often agents fail or require clarification.

By instrumenting the agents (e.g. capturing traces with LangGraph or using LangSmith/DeepEval), you can quantitatively evaluate and improve them. This project gives you hands-on practice with agent evaluation metrics [\[33†L51-L59\]](#) [\[35†L148-L158\]](#) .

Tokenization & Latency Optimization

Token optimization: LLM calls cost both time and money per token. Apply prompt engineering to minimize tokens:

- **Concise prompts:** Use short system prompts and dynamic templates.
- **Context pruning:** Only include relevant context in RAG queries; drop extraneous text.
- **Hybrid models:** For simple tasks (like routing or short replies), use a smaller model (GPT-4o or LLaMA) and reserve GPT-4v for complex analysis.
- **Caching:** Cache frequent queries and RAG results. Tools like Redis or LangCache can store embeddings/LLM outputs. This yields huge speed-ups on repeated queries [\[37†L314-L323\]](#) . Redis's blog notes that semantic caching can cut repeated-LM calls by ~73%, returning answers in milliseconds instead of seconds [\[37†L314-L323\]](#) .

Latency reduction:

- **Parallel calls:** When possible, query Jira, Confluence, and the DB in parallel.
- **Asynchronous I/O:** Use async requests to APIs.
- **Limit output size:** Because output tokens are generated sequentially, keep LLM outputs succinct. The Redis article shows that every output token adds latency, so halving token output often halves time [\[37†L273-L282\]](#) .
- **Optimize token use:** According to Redis, "LLM token optimization minimizes token consumption... to reduce API costs and improve inference latency" [\[37†L273-L282\]](#) . In practice, trim prompt verbosity and use tokens economically (e.g. abbreviate system instructions, split large tasks).

By actively monitoring token counts and response times, you'll practice profiling performance. The project naturally forces you to optimize prompts and maybe implement things like streaming or incremental answer generation to keep latency low.

Model & Agent Evaluation Skills

This project directly exercises **evaluation and metrics** skills:

- You can use frameworks like LangSmith or DeepEval to build automated tests (unit tests for agents). For example, create test prompts with known answers (e.g. "Who is in Sales?") and measure correctness. Use agent-level metrics (plan/tool/step metrics [\[33†L51-L59\]](#)) to diagnose failures.
- RAG evaluation comes into play when assessing how well the knowledge agent finds relevant documents [\[35†L148-L158\]](#) . You can measure retrieval metrics (precision/recall) and answer faithfulness.

Additionally, optimizing token usage and latency is a critical skill. You'll learn to profile your system, use caching, and choose LLM models strategically. The task explicitly involves connecting to an MCP server

(Atlassian's Rovo or an open MCP), which gives experience in real-world LLM-tool integration [39†L48-L51] [13†L261-L264] .

MCP Server Integration

You will indeed implement an **MCP connection**. For example, configure the Atlassian Rovo MCP server in your agent (via `claude mcp add atlassian` or setting the config URL [39†L48-L51] . This lets your LLM directly call Atlassian tools: e.g. "list open PRs in repo X" triggers the `bitbucketPullRequest.list` MCP tool [13†L261-L264] .

Alternatively, you might run the open-source `mcp-atlassian` bridge and point your agent to it [11†L17-L25] . In either case, you'll gain hands-on experience with the Model Context Protocol standard and OAuth flows for enterprise tools.

Tools & Stack (example)

- **Orchestration:** LangGraph, AgentFlow, or a custom orchestration layer in Python/TypeScript.
- **LLMs:** GPT-4o/GPT-4v, Claude, or open models (for lower-cost tasks).
- **Vector DB:** Pinecone, Chroma, or RedisVector for RAG.
- **Database:** PostgreSQL (with a suitable schema for employees, etc.).
- **Atlassian APIs:** Use Atlassian's REST endpoints (with API tokens) or the Atlassian Rovo MCP (via `mcp.atlassian.com`).
- **Backend:** FastAPI or similar to glue everything and handle webhooks/auth.
- **Monitoring/Eval:** LangSmith, DeepEval or custom logging for agent traces.
- **Caching/Memory:** Redis (for hot RAG results, embeddings cache, etc.).

Summary

This end-to-end project demonstrates an **enterprise-grade agentic system**: it uses multiple agents, external tool integration (Atlassian, SQL DB), RAG, memory, and eval. It naturally involves all your target skills:

- **Model & Agent Evaluation:** You'll build in metrics and test cases to measure each agent's correctness and plan/execution quality [33†L51-L59] [35†L148-L158] .
- **Tokenization & Latency:** Optimizing prompts and response length is essential to keep the system fast and cost-effective [37†L273-L282] . Caching repeated queries will also be key [37†L314-L323] .
- **MCP Integration:** Connecting to Atlassian via MCP (or using their APIs) is central to the Performance Agent. You'll learn the OAuth/token setup and how an LLM can call these tools natively [39†L48-L51] [13†L261-L264] .

Overall, the project outline above covers the system architecture, data sources, and workflows. It ensures you'll practice agent orchestration, tool calling, RAG design, guardrails/HITL, and the performance optimizations that companies look for in AI solutions.

Sources: Atlassian's docs on MCP and APIs [39†L48-L51] [13†L261-L264] [27†L24-L32] [26†L1699-L1707] , Redis on token/latency optimization [37†L273-L282] [37†L314-L323] , and evaluation best practices [33†L51-L59] [35†L148-L158] .

